Solving N-Queens with SAT: An Experimental Comparison of Cardinality Encodings and Exact Solvers

Do Duc Long[a,*]

[a]VNU University of Engineering and Technology, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-queens
SAT encoding
at-most-one constraint
constraint programming
integer linear programming

ABSTRACT

The N-queens problem is a classical benchmark for exact solvers. This paper compares how well SAT solvers handle it under five at-most-one (AMO) encodings, two formulations, and four SAT solvers, and then compares the best SAT configuration with CPLEX CP Optimizer, CPLEX MIP, Gurobi, and CP-SAT on boards up to 1024×1024. Each configuration is run once with a time limit of 180 s on a shared laptop, so the results are indicative rather than statistically rigorous. The commander encoding with the minimal formulation was the best SAT configuration (PAR-2 1.38 s), CaDiCaL was the best SAT solver, and CPLEX CP and Gurobi were the only methods that solved every board, including $n = 1024$.

1. Introduction

The N-queens problem asks for a placement of n queens on an $n \times n$ chessboard such that no two queens share a row, a column, or a diagonal. Since its introduction as a chess puzzle, it has become a standard benchmark for constraint satisfaction, search, and SAT encodings [2]. Explicit constructions place n queens for every $n \geq 4$, so a single placement can be obtained in polynomial time; in contrast, the completion variant with pre-placed queens is NP-complete [8]. The empty-board problem nevertheless remains a useful testbed for exact solvers: its $6n - 2$ row, column, and diagonal constraints overlap on every square, their lengths range from 2 to n, and the model grows quadratically with n.

Three exact paradigms are commonly applied to such problems. Constraint programming (CP) states the problem with three all-different constraints over n integer variables and relies on dedicated propagators [21, 14, 18]. Integer linear programming (ILP) uses n^2 binary variables and linear constraints solved by LP-based branch and cut [9, 11]. Boolean satisfiability (SAT) uses the same binary variables but requires every constraint to be written in conjunctive normal form (CNF). In SAT, each row, column, and diagonal becomes an at-most-one (AMO) constraint, and the CNF encoding of AMO determines both the size of the formula and the strength of unit propagation [19]. Many AMO encodings have been proposed, including the pairwise, binary [7], sequential counter [22], commander [13], and product [4] encodings, and their relative performance is known to depend on the problem [6, 17].

However, for N-queens, the interaction between the AMO encoding, the choice of redundant constraints, and the SAT solver has not been reported under a common protocol, and the comparison with commercial CP and ILP solvers on large boards is likewise missing. This paper addresses this gap with an experimental study of SAT-based solving for N-queens on boards up to 1024×1024. The main contributions of this paper are as follows.

1. A common experimental protocol for N-queens that compares five AMO encodings (pairwise, binary, sequential counter, commander, product) in two formulations, with every returned placement checked by an independent validator.
2. A comparison of four SAT solvers (CaDiCaL, Glucose, MiniSat, MapleChrono) on the best encoding.
3. A comparison of the resulting SAT configuration with CPLEX CP, CPLEX MIP, Gurobi, and CP-SAT on boards up to 1024×1024.

The remainder of the paper is organized as follows. Section 2 defines the problem, presents the ILP and CP reference models, and reviews related work. Section 3 presents the SAT encoding, the five AMO encodings, and their size analysis. Section 4 describes the experimental setup, and Section 5 reports the computational results. Section 6 concludes the paper.

*Corresponding author
Email address: 24021550@vnu.edu.vn (D.D. Long)

2. Problem definition and related work

2.1. Problem definition

Let $I = \{0, \dots, n-1\}$ index the rows and columns of the board. Because every row contains exactly one queen, a placement is a vector $q = (q_0, \dots, q_{n-1})$ in which $q_r \in I$ is the column of the queen in row r. A placement is feasible if and only if

$$q_r \neq q_s \qquad \forall r < s, \tag{1}$$

$$q_r - r \neq q_s - s \qquad \forall r < s, \tag{2}$$

$$q_r + r \neq q_s + s \qquad \forall r < s. \tag{3}$$

Constraint (1) forbids two queens in one column, and Constraints (2) and (3) forbid two queens on one descending or ascending diagonal. The problem has no objective function: the task is to find one feasible placement. Feasible placements exist for $n = 1$ and every $n \geq 4$, whereas $n = 2$ and $n = 3$ are infeasible. Figure 1 shows a placement for $n = 8$ returned in our experiments.

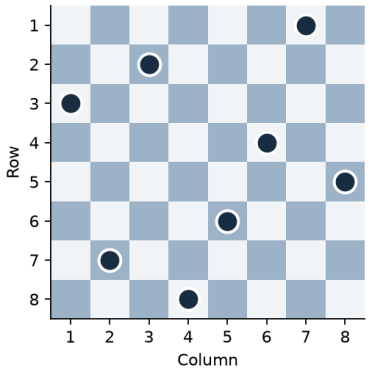


Figure 1: A solution of the 8×8 instance returned by the SAT solver and accepted by the independent checker.

2.2. ILP and CP formulations

The ILP model uses a binary variable x_{rc} for every square $(r, c) \in I^2$, with $x_{rc} = 1$ if and only if a queen occupies the square. Let $D_d^- = \{(r, c) : r - c = d\}$ and $D_s^+ = \{(r, c) : r + c = s\}$ denote the descending and ascending diagonals. The model is

$$\sum_{c \in I} x_{rc} = 1 \qquad \forall r \in I, \tag{4}$$

$$\sum_{r \in I} x_{rc} = 1 \qquad \forall c \in I, \tag{5}$$

$$\sum_{(r,c) \in D_d^-} x_{rc} \leq 1 \qquad \forall d : |D_d^-| \geq 2, \tag{6}$$

$$\sum_{(r,c) \in D_s^+} x_{rc} \leq 1 \qquad \forall s : |D_s^+| \geq 2, \tag{7}$$

$$x_{rc} \in \{0, 1\} \qquad \forall (r, c) \in I^2, \tag{8}$$

with a constant objective. For $n \geq 2$ the model has n^2 variables and $6n - 6$ constraints, because the four single-square diagonals need no constraint. Gurobi and CPLEX MIP solve the identical model.

The CP model uses the integer variables $q_r \in I$ of the problem definition and states Constraints (1)–(3) as three global constraints:

$$\text{AllDifferent}(q_r)_{r \in I}, \quad \text{AllDifferent}(q_r + r)_{r \in I}, \quad \text{AllDifferent}(q_r - r)_{r \in I}. \tag{9}$$

CPLEX CP Optimizer and OR-Tools CP-SAT solve the identical model. No model receives symmetry-breaking constraints, a warm start, or a precomputed placement.

2.3. Related work

N-queens. Bell and Stevens [2] survey the results on the problem, including explicit constructions for every $n \geq 4$ and its many variants. Gent et al. [8] proved that N-queens completion is NP-complete and that counting its completions is #P-complete. The empty-board problem is therefore studied here as a structured benchmark for exact solvers rather than as a hard decision problem.

At-most-one encodings. Prestwich [19] reviews CNF encodings of finite-domain and cardinality constraints. The pairwise encoding needs no auxiliary variables but a quadratic number of clauses. The binary encoding of Frisch and Peugniez [7] uses a logarithmic number of auxiliary variables. Sinz [22] introduced the sequential counter, whose AMO case has linear size and maintains arc consistency by unit propagation. Klieber and Kwon [13] proposed the commander encoding, which applies AMO hierarchically to groups of variables, and Chen [4] proposed the product encoding, which maps the variables onto a two-dimensional grid. Nguyen and Mai [17] combined the binary and commander ideas in the bimander encoding. For sequences of overlapping AMO constraints, Truong et al. [23] share sequential-counter sub-formulae between consecutive constraints and reduce both variables and clauses. Comparative studies [6] report that no single encoding is best across problems, which motivates an evaluation on the problem at hand.

Exact solvers. Modern SAT solvers follow the conflict-driven clause learning (CDCL) scheme introduced by GRASP [15]. The SAT solvers evaluated here are MiniSat [5], Glucose, which ranks learned clauses by literal block distance [1], CaDiCaL [3], and MapleChrono, which adds chronological backtracking [16]; all four are accessed through PySAT [12]. In CP, the all-different constraint has a filtering algorithm based on bipartite matching [21]. CPLEX CP Optimizer [14] is a CP engine with automatic search, and OR-Tools CP-SAT [18] combines CP propagators with a CDCL SAT core. Gurobi [9] and CPLEX [11] solve ILP models by LP-based branch and cut.

3. SAT encoding for the N-queens problem

3.1. Encoding overview

The SAT encoding uses the same occupancy variables x_{rc} as the ILP model, numbered $rn + c + 1$ in DIMACS format. Table 1 summarizes the notation. For a set of literals X, let

$$\mathrm{ALO}(X) = \bigvee_{x \in X} x, \qquad \mathrm{EO}(X) = \mathrm{ALO}(X) \wedge \mathrm{AMO}(X), \tag{10}$$

where $\mathrm{AMO}(X)$ is the CNF produced by one of the AMO encodings in Section 3.2. Let R_r, C_c, and $\mathcal{D}$ denote the rows, the columns, and the set of diagonals with at least two squares. We consider two formulations. The *minimal* formulation

$$F_{\min} = \bigwedge_{r \in I} \mathrm{EO}(R_r) \;\wedge\; \bigwedge_{c \in I} \mathrm{AMO}(C_c) \;\wedge\; \bigwedge_{D \in \mathcal{D}} \mathrm{AMO}(D) \tag{11}$$

contains only the constraints needed for correctness, and the *extended* formulation

$$F_{\mathrm{ext}} = F_{\min} \wedge \bigwedge_{c \in I} \mathrm{ALO}(C_c) \tag{12}$$

adds n redundant column clauses and no variables. The redundant clauses mirror the column equality (5) of the ILP model and make column occupancy directly available to unit propagation.

Proposition 1. *For every $n \geq 1$ and every correct AMO encoding, the projections of the models of $F_{\min}$ and of F_{ext} onto the variables x_{rc} are exactly the feasible placements of Constraints (1)–(3).*

The proof is given in Appendix A. The solution is decoded from the n^2 board variables only, so auxiliary variables cannot create queens.

Table 1
Notation.

Symbol	Description
n	Board side; the number of queens
I	Row and column indices $\{0, \dots, n-1\}$
x_{rc}	Boolean variable: a queen occupies square (r, c)
R_r, C_c	Literals of row r and column c
D	Diagonals with at least two squares ($4n-6$ for $n \geq 2$)
ALO, AMO, EO	At-least-one, at-most-one, exactly-one constraints
k	Number of inputs of one AMO constraint
$a_E(k)$, $c_E(k)$	Auxiliary variables and clauses of encoding E for k inputs
F_{min}, F_{ext}	Minimal and extended formulations

3.2. At-most-one encodings

Let $X = (x_1, \dots, x_k)$ be the inputs of one AMO constraint. An encoding $E(X, Y)$ with auxiliary variables Y is correct if $\text{AMO}(X) \Leftrightarrow \exists Y\ E(X, Y)$. Fresh auxiliary variables are created for every AMO constraint, and constraints with $k \leq 1$ produce no clauses. The pairwise, commander, and product encodings are implemented directly; the binary and sequential counter encodings are generated by PySAT [20].

Pairwise. The pairwise encoding forbids every pair of true inputs:

$$E_{\text{pair}}(X) = \bigwedge_{1 \leq i < j \leq k} (\neg x_i \vee \neg x_j). \tag{13}$$

It uses no auxiliary variables and $\binom{k}{2}$ clauses.

Binary. The binary encoding [7] introduces $b = \lceil \log_2 k \rceil$ bits $y_0, \dots, y_{b-1}$ and assigns input x_i the binary representation of $i-1$. Let ℓ_{it} be y_t if bit t of that representation is one and $\neg y_t$ otherwise. Then

$$E_{\text{bin}}(X, Y) = \bigwedge_{i=1}^{k} \bigwedge_{t=0}^{b-1} (\neg x_i \vee \ell_{it}). \tag{14}$$

A true input fixes all bits, which falsifies every other input. The encoding uses b auxiliary variables and kb clauses for $k \geq 3$; PySAT encodes $k = 2$ with one pairwise clause.

Commander. The commander encoding [13] partitions X into $m = \lceil k/3 \rceil$ consecutive groups $G_1, \dots, G_m$ of at most three inputs and introduces a commander variable a_j per group:

$$E_{\text{cmd}}(X, A) = \bigwedge_{j=1}^{m} \left[E_{\text{pair}}(G_j) \wedge \bigwedge_{x \in G_j} (\neg x \vee a_j) \wedge \left(\neg a_j \vee \bigvee\nolimits_{x \in G_j} x \right) \right] \wedge E_{\text{cmd}}(A). \tag{15}$$

The commanders are constrained recursively, and the recursion stops with the pairwise encoding when $k \leq 4$. A true input forces its commander, which excludes the other commanders and hence the inputs of other groups.

Product. The product encoding [4] places the inputs on a $p \times q$ grid with $p = \lceil \sqrt{k} \rceil$ and $q = \lceil k/p \rceil$. Input x_i occupies row $u(i) = \lfloor (i-1)/q \rfloor + 1$ and column $v(i) = ((i-1) \bmod q) + 1$ of the grid, and

$$E_{\text{prod}}(X, U, V) = \bigwedge_{i=1}^{k} \left[(\neg x_i \vee u_{u(i)}) \wedge (\neg x_i \vee v_{v(i)}) \right] \wedge E_{\text{prod}}(U) \wedge E_{\text{prod}}(V). \tag{16}$$

Two distinct inputs differ in a grid row or a grid column and therefore violate one of the recursive AMO constraints. The recursion stops with the pairwise encoding when $k \leq 4$.

Sequential counter. The sequential counter [22] introduces prefix variables $s_1, \dots, s_{k-1}$, where s_i is true if one of $x_1, \dots, x_i$ is true:

$$(\neg x_1 \vee s_1) \wedge (\neg x_k \vee \neg s_{k-1}) \wedge \bigwedge_{i=2}^{k-1} \left[(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1}) \right]. \tag{17}$$

It uses $k-1$ auxiliary variables and $3k-4$ clauses for $k \geq 3$.

For all five encodings, setting one input to true falsifies all other inputs of the same constraint by unit propagation alone. Appendix A describes how this property and the correctness of every encoding were verified.

3.3. Encoding size analysis

Let $a_E(k)$ and $c_E(k)$ be the auxiliary variables and clauses of encoding E for one constraint with $k \geq 2$ inputs. Pairwise gives $(0, \binom{k}{2})$, binary $(\lceil \log_2 k \rceil, k\lceil \log_2 k \rceil)$, and sequential counter $(k-1, 3k-4)$, where binary and sequential counter use $(0, 1)$ for $k = 2$. For commander and product, $k \leq 4$ uses the pairwise counts, and larger constraints follow the recurrences

$$a_{\text{cmd}}(k) = m + a_{\text{cmd}}(m), \qquad c_{\text{cmd}}(k) = \textstyle\sum_{j=1}^{m} \binom{|G_j|}{2} + k + m + c_{\text{cmd}}(m), \tag{18}$$

$$a_{\text{prod}}(k) = p + q + a_{\text{prod}}(p) + a_{\text{prod}}(q), \qquad c_{\text{prod}}(k) = 2k + c_{\text{prod}}(p) + c_{\text{prod}}(q). \tag{19}$$

For $n \geq 2$ the board has $2n+2$ constraints of length n (rows, columns, and the two main diagonals) and four constraints of each length $k = 2, \ldots, n-1$. The total number of variables and clauses of the minimal formulation is therefore

$$V_E(n) = n^2 + (2n+2)\, a_E(n) + 4 \sum_{k=2}^{n-1} a_E(k), \tag{20}$$

$$C_{E,\min}(n) = n + (2n+2)\, c_E(n) + 4 \sum_{k=2}^{n-1} c_E(k), \tag{21}$$

where the first term n counts the row ALO clauses. The extended formulation has the same variables and $C_{E,\min}(n)+n$ clauses. For the pairwise encoding, Equation (21) evaluates to $C_{\text{pair},\min}(n) = (5n^3 - 6n^2 + 4n)/3$. Table 2 summarizes the asymptotic sizes, and Figure 2 plots the exact counts. Equations (20) and (21) were checked against the generated CNF for every board size used in the experiments.

Table 2
Asymptotic size of the complete board CNF.

Encoding	Auxiliary variables	Clauses
Pairwise	0	$\Theta(n^3)$
Binary	$\Theta(n \log n)$	$\Theta(n^2 \log n)$
Commander	$\Theta(n^2)$	$\Theta(n^2)$
Product	$\Theta(n^{3/2})$	$\Theta(n^2)$
Sequential counter	$\Theta(n^2)$	$\Theta(n^2)$

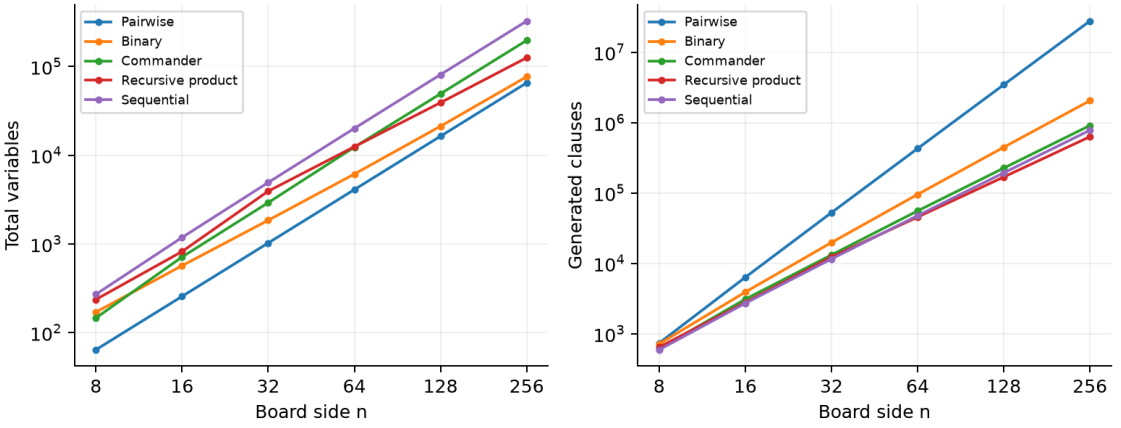


Figure 2: Total variables (left) and clauses (right) of the minimal formulation as a function of the board side n, on logarithmic axes.

For $n = 256$ the pairwise encoding generates 27,831,296 clauses, whereas the commander, product, and sequential counter encodings generate 911,754, 631,464, and 780,552 clauses. The binary encoding uses the fewest variables (76,784) but more clauses (2,053,204) than the other non-pairwise encodings.

4. Experimental setup

4.1. Implementation

The SAT encoder, the CP and ILP models, and the experiment harness are written in Python. The encoder streams clauses directly into the SAT solver and can also export the formula in DIMACS format. The CP model is passed to CPLEX CP Optimizer through DOcplex [10] and to CP-SAT through the OR-Tools Python interface; the ILP model is passed to Gurobi and CPLEX through their Python interfaces. Every returned placement is checked by an independent validator for its length, domain, and uniqueness of columns and of both diagonal indices.

4.2. Benchmark instances and experiments

The benchmark consists of the empty $n \times n$ boards listed in Table 3. Three experiments are performed. Experiment E1 compares the ten combinations of five AMO encodings and two formulations using CaDiCaL. Experiment E2 fixes the best configuration of E1 and compares four SAT solvers. Experiment E3 compares the resulting SAT configuration with CPLEX CP, CPLEX MIP, Gurobi, and CP-SAT and extends the board side to $n = 1024$.

Table 3
Summary of the experiments. Each configuration is run once on each board.

Experiment	Board sides n	Methods	Runs
E1: AMO encodings	8, 16, 32, 64, 128, 256	5 encodings × 2 formulations, CaDiCaL	60
E2: SAT solvers	8, 16, 32, 64, 128, 256	CaDiCaL, Glucose, MiniSat, MapleChrono	24
E3: exact solvers	8, 16, . . . , 256, 512, 1024	SAT, CPLEX CP, CPLEX MIP, Gurobi, CP-SAT	40

4.3. Environment and metrics

All experiments were conducted on a laptop with an AMD Ryzen 7 8845HS processor (8 cores, 16 threads) and 32 GB of RAM, running Windows 11 Home. We used Python 3.14, PySAT 1.9.dev2 with CaDiCaL 1.9.5, Glucose 4.2, MiniSat 2.2, and MapleChrono, Gurobi 13.0.3, IBM ILOG CPLEX Optimization Studio 22.2 (CPLEX MIP and CP Optimizer), and OR-Tools 9.15. Every solver uses one thread and random seed 1. Each run is executed in a fresh process with a time limit of 180 s, which covers library imports, license acquisition, model construction, and solving.

A run that exceeds the limit is recorded as a timeout (TO). The experiments were run sequentially, one solver process at a time.

We report three metrics. *Process time* is the wall-clock time of the complete run and is the primary metric. *Core time* is the time for model construction and solving only, which excludes imports and license acquisition. *PAR-2* is the mean process time over all runs of a method, where each timeout counts as twice the time limit (360 s). Because each configuration is run once, tables report that single measurement and TO marks a timeout. The laptop was shared with other jobs, so absolute times carry noise and only large differences should be read as real.

5. Experimental results

5.1. AMO encodings

Table 4 ranks the ten configurations of Experiment E1 by PAR-2. The commander encoding with the minimal formulation is best (1.38 s), followed by the commander encoding with the extended formulation (1.95 s). The pairwise encoding timed out at $n = 256$ because its formula has 27,831,296 clauses. The product and sequential counter encodings in the minimal formulation also timed out at that size, while their extended variants solved it in 17.73 s and 126.0 s. The redundant column ALO clauses of the extended formulation therefore helped the product, sequential counter, and binary encodings, but did not help the commander encoding, whose minimal formulation was faster (4.51 s against 8.17 s). Figure 3 shows the PAR-2 time as a function of n.

Table 4
Experiment E1: encodings ranked by PAR-2 over six board sizes with CaDiCaL.

Encoding / formulation	Solved	PAR-2 (s)	Variables	Clauses
commander/minimal	6/6	1.38	197,342	911,754
commander/extended	6/6	1.95	197,342	912,010
binary/extended	6/6	4.15	76,784	2,053,460
product/extended	6/6	4.53	125,800	631,720
binary/minimal	6/6	16.33	76,784	2,053,204
seqcounter/extended	6/6	22.92	326,142	780,808
pairwise/minimal	5/6	62.05	65,536	27,831,296
pairwise/extended	5/6	62.21	65,536	27,831,552
seqcounter/minimal	5/6	71.85	326,142	780,552
product/minimal	4/6	122.8	125,800	631,464

5.2. SAT solvers

Table 5 compares the solvers on the commander encoding with the minimal formulation. CaDiCaL solved all boards (PAR-2 2.91 s) and MiniSat solved all of them but was slower at large n. Glucose (4/6 solved) and MapleChrono (3/6 solved) timed out on the larger boards.

Table 5
Experiment E2: process time (s) of four SAT solvers.

n	CaDiCaL	Glucose	MiniSat	MapleChrono
8	0.82	1.32	0.81	0.81
16	0.77	0.69	0.75	24.00
32	0.99	48.00	0.65	31.07
64	0.86	20.21	0.81	TO
128	2.24	TO	13.95	TO
256	11.76	TO	13.42	TO
Solved	6/6	4/6	6/6	3/6
PAR-2	2.91	131.7	5.07	189.3

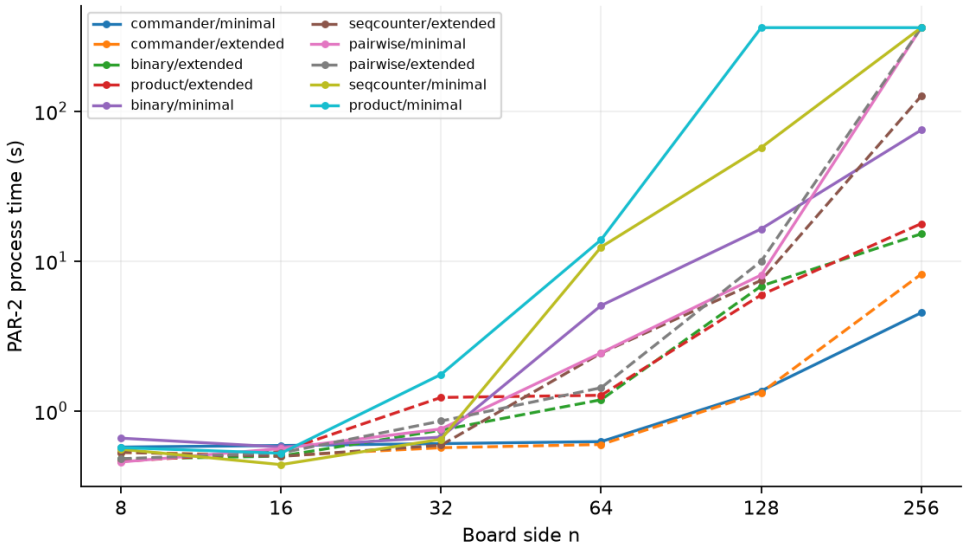


Figure 3: PAR-2 process time of the ten SAT configurations.

5.3. Comparison with CP and ILP solvers

Table 6 and Figure 4 compare the SAT configuration with the CP and ILP solvers. Up to $n = 64$ all methods finish within a few seconds, and the differences come mostly from start-up cost. From $n = 128$ the solvers separate. CPLEX CP and Gurobi solved every board, including $n = 1024$ in 10.21 s and 19.76 s. The SAT configuration needed 71.47 s at $n = 512$ and timed out at $n = 1024$, as did CPLEX MIP. CP-SAT was the slowest, taking 77.52 s at $n = 256$ and timing out beyond it.

Table 6
Experiment E3: process time (s) of the SAT configuration and the CP and ILP solvers.

n	SAT	CPLEX CP	CPLEX MIP	Gurobi	CP-SAT
8	0.63	3.19	0.76	0.75	2.65
16	0.58	3.06	0.74	0.65	2.78
32	0.62	3.07	0.79	0.70	2.85
64	0.76	3.11	0.90	0.66	2.97
128	1.61	2.52	1.09	1.22	8.36
256	10.03	3.20	3.92	1.62	77.52
512	71.47	5.77	12.65	5.21	TO
1024	TO	10.21	TO	19.76	TO
Solved	7/8	8/8	7/8	8/8	6/8
PAR-2	55.71	4.26	47.61	3.82	102.1

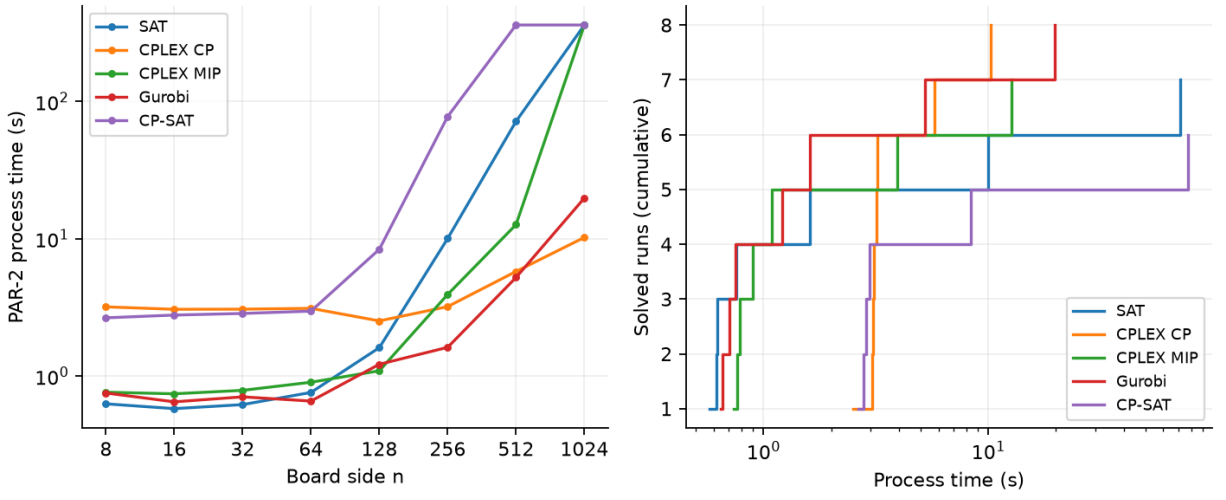


Figure 4: PAR-2 time against board side (left) and cumulative solved runs (right).

5.4. Discussion

The choice of AMO encoding matters more than the choice between the two formulations: the gap between the best and worst encodings at $n = 256$ is a timeout against a few seconds, and the encodings that add auxiliary variables hierarchically (commander, product) keep the clause count near $\Theta(n^2)$ while the pairwise encoding grows as $\Theta(n^3)$. On this problem a dedicated CP propagator or an LP-based solver still beats the best SAT model on very large boards, because the SAT formula grows quadratically. Threats to validity are the single run per configuration, the shared machine, and the 180 s limit, which turns slow runs into censored observations.

6. Conclusion

We compared five AMO encodings, two formulations, four SAT solvers, and four CP and ILP solvers on the N-queens problem. The commander encoding with the minimal formulation and CaDiCaL was the best SAT configuration, and the extended formulation helped all encodings except commander. CPLEX CP and Gurobi were the only methods to solve every board up to $n = 1024$. The results come from one run per configuration on a shared laptop, so they show broad trends rather than precise rankings. A natural extension is repeated runs on a dedicated machine.

CRediT authorship contribution statement

Do Duc Long: Conceptualization, Methodology, Software, Validation, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code, raw results, returned placements, and report sources are available at `https://github.com/dolong226/homework1-nqueens`.

A. Correctness of the SAT encoding

Proof of Proposition 1. Let σ be a model of $F_{\min}$. The row constraints $\mathrm{EO}(R_r)$ place exactly one queen in each row, so σ defines a vector q with q_r equal to the column of the queen in row r. The column constraints $\mathrm{AMO}(C_c)$ forbid two of these n queens in one column, which gives Constraint (1), and the diagonal constraints give Constraints (2) and (3). Conversely, a feasible placement satisfies every ALO and AMO constraint on the board variables, and each correct AMO encoding can extend it to its auxiliary variables. The same placement also satisfies the column ALO clauses of F_{ext}, because n queens in n distinct columns occupy every column. Hence both formulations have the same projected models. □

Correctness of the AMO encodings. Each encoding is correct for the following reasons. In the pairwise encoding, two true inputs violate their clause. In the binary encoding, two true inputs require different bit patterns. In the commander encoding, two true inputs in one group violate a pairwise clause, and two true inputs in different groups force two commanders. In the product encoding, two true inputs force two grid rows or two grid columns. In the sequential counter, a true input x_i forces $s_i, s_{i+1}, \dots$ and thereby falsifies every later input. Conversely, an assignment with at most one true input is extended by setting each auxiliary variable to the disjunction of the inputs it represents, and the all-false assignment is extended by setting all auxiliary variables to false.

Automated verification. The implementation was verified by three test suites. First, every truth assignment of every AMO encoding was enumerated for $k = 0, \dots, 11$ inputs with positive and negative literals. Second, all solutions of the ten SAT configurations were enumerated for $n = 1, \dots, 8$ with blocking clauses over the board variables only and compared with an independent backtracking solver; the counts $1, 0, 0, 2, 10, 4, 40, 92$ match. Third, for $k = 1, \dots, 64$, setting each input to true was shown to falsify all other inputs by unit propagation alone. The CP and ILP models were checked on the infeasible board $n = 3$ and the feasible board $n = 8$.

B. Detailed encoding results

Table 7
Median process time (s) of each encoding and formulation with CaDiCaL on each board size (Experiment E1).

Encoding / formulation	$n = 8$	$n = 16$	$n = 32$	$n = 64$	$n = 128$	$n = 256$
pairwise/minimal	0.46	0.56	0.76	2.44	8.06	TO
binary/minimal	0.66	0.57	0.67	5.04	16.35	74.71
commander/minimal	0.57	0.59	0.60	0.62	1.36	4.51
product/minimal	0.57	0.52	1.75	13.86	TO	TO
seqcounter/minimal	0.55	0.44	0.65	12.38	57.08	TO
pairwise/extended	0.48	0.52	0.86	1.43	9.95	TO
binary/extended	0.48	0.50	0.74	1.19	6.80	15.16
commander/extended	0.53	0.50	0.57	0.60	1.32	8.17
product/extended	0.46	0.53	1.23	1.27	5.94	17.73
seqcounter/extended	0.53	0.50	0.59	2.43	7.42	126.0

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used OpenAI Codex and Claude Code (Anthropic) to assist with implementation, experiment automation, and editing of the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence (IJCAI 2009), pp. 399–404.

[2] Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31. doi:10.1016/j.disc.2007.12.043.

[3] Biere, A., Fazekas, K., Fleury, M., Heisinger, M., 2020. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020, in: Proceedings of SAT Competition 2020 – Solver and Benchmark Descriptions, University of Helsinki. pp. 51–53.

[4] Chen, J., 2010. A new SAT encoding of the at-most-one constraint. Workshop on Constraint Modelling and Reformulation (ModRef).

[5] Eén, N., Sörensson, N., 2004. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing – SAT 2003, pp. 502–518. doi:10.1007/978-3-540-24605-3_37.

[6] Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most-k constraint: Some old, some new, some fast, some slow. Workshop on Constraint Modelling and Reformulation (ModRef).

[7] Frisch, A.M., Peugniez, T.J., 2001. Solving non-Boolean satisfiability problems with stochastic local search, in: Proceedings of the 17th International Joint Conference on Artificial Intelligence (IJCAI 2001), pp. 282–288.

[8] Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-queens completion. Journal of Artificial Intelligence Research 59, 815–848. doi:10.1613/jair.5512.

[9] Gurobi Optimization, n.d. Gurobi optimizer reference manual. https://docs.gurobi.com/projects/optimizer/en/current/. Version 13.0.3 used in the experiments.

[10] IBM, n.d.a. DOcplex constraint programming model documentation. https://ibmdecisionoptimization.github.io/docplex-doc/cp/docplex.cp.model.py.html. Accessed 10 October 2026.

[11] IBM, n.d.b. IBM ILOG CPLEX optimization studio. https://www.ibm.com/products/ilog-cplex-optimization-studio. Version 22.2 used in the experiments.

[12] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, pp. 428–437. doi:10.1007/978-3-319-94144-8_26.

[13] Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects. Workshop on Constraints in Formal Verification. URL: https://www.cs.cmu.edu/~wklieber/papers/2007_efficient-cnf-encoding-for-selecting-1.pdf.

[14] Laborie, P., Rogerie, J., Shaw, P., Vilím, P., 2018. IBM ILOG CP optimizer for scheduling. Constraints 23, 210–250. doi:10.1007/s10601-018-9281-x.

[15] Marques-Silva, J.P., Sakallah, K.A., 1999. GRASP: A search algorithm for propositional satisfiability. IEEE Transactions on Computers 48, 506–521. doi:10.1109/12.769433.

[16] Nadel, A., Ryvchin, V., 2018. Chronological backtracking, in: Theory and Applications of Satisfiability Testing – SAT 2018, pp. 111–121. doi:10.1007/978-3-319-94144-8_7.

[17] Nguyen, V.H., Mai, S.T., 2015. A new method to encode the at-most-one constraint into SAT, in: Proceedings of the Sixth International Symposium on Information and Communication Technology (SoICT 2015), pp. 46–53.

[18] Perron, L., Didier, F., n.d. CP-SAT. Google OR-Tools, https://developers.google.com/optimization/cp/cp_solver/. Version 9.15 used in the experiments.

[19] Prestwich, S., 2021. CNF encodings, in: Biere, A., Heule, M., van Maaren, H., Walsh, T. (Eds.), Handbook of Satisfiability. 2 ed.. IOS Press, pp. 75–100.

[20] PySAT Developers, n.d. Cardinality encodings: pysat.card. https://pysathq.github.io/docs/html/api/card.html. Accessed 10 October 2026.

[21] Régin, J.C., 1994. A filtering algorithm for constraints of difference in CSPs, in: Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94), pp. 362–367.

[22] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, pp. 827–831. doi:10.1007/11564751_73.

[23] Truong, H.X., Kieu, T.V., To, K.V., 2025. Sequential counter encoding for staircase at-most-one constraints, in: Proceedings of the 17th International Conference on Agents and Artificial Intelligence (ICAART 2025), pp. 164–175. doi:10.5220/0013124600003890.